

การทดลองที่ 5 BCD 7 Segment & Counter

วัตถุประสงค์

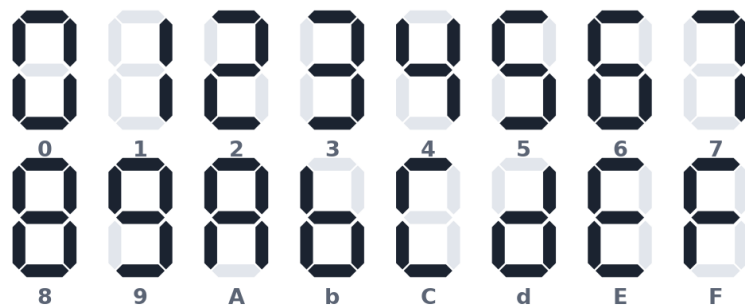
1. เพื่อให้สามารถออกแบบวงจรนับแบบ Asynchronous และ Synchronous ได้
2. เพื่อให้นักศึกษาออกแบบวงจร MOD ความถี่ได้
3. เพื่อให้สามารถสร้างวงจรนับที่ออกแบบโดยใช้ JK Flip Flop ได้

การทดลอง

ก่อนดำเนินการทดลองแต่ละข้อ ให้นักศึกษาศึกษา **กล่องหลักการ พร้อมรูปประกอบ** เพื่อทบทวนแนวคิดและขั้นตอนการออกแบบ จากนั้นจึงลงมือทำโจทย์ในแต่ละข้อ

1. ให้นักศึกษาออกแบบวงจร BCD (HEX) to 7-Segment สำหรับถอดรหัสเลขฐานสองขนาด 4 บิต เพื่อขับสัญญาณให้ตัว 7-Segment แสดงผลเลข 0 – F ดังรูปที่ 1

หมายเหตุ: หลักการออกแบบ BCD / HEX to 7-Segment ได้เรียนแล้วในการทดลองที่ 4 นักศึกษาสามารถอ้างอิงวิธีการและตารางค่าความจริงจากการทดลองดังกล่าว

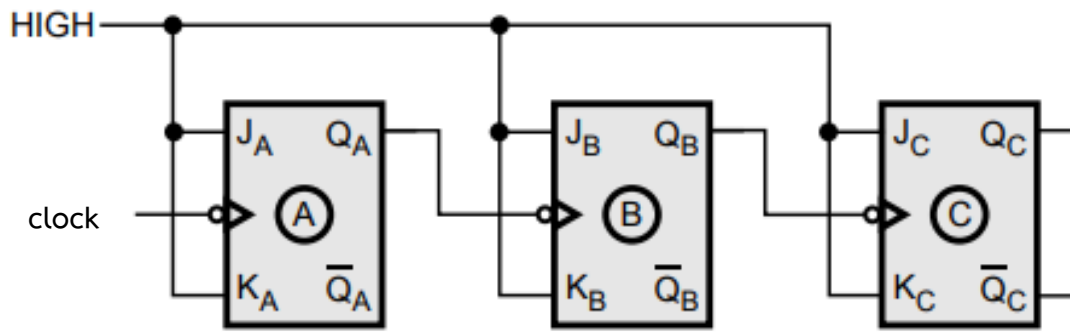


รูปที่ 1 รูปแบบการแสดงผลเลข 0-F บน 7-Segment ที่ต้องออกแบบ

หลักการวงจรหารความถี่ (MOD) และวงจรนับ 0-7 แบบ Asynchronous

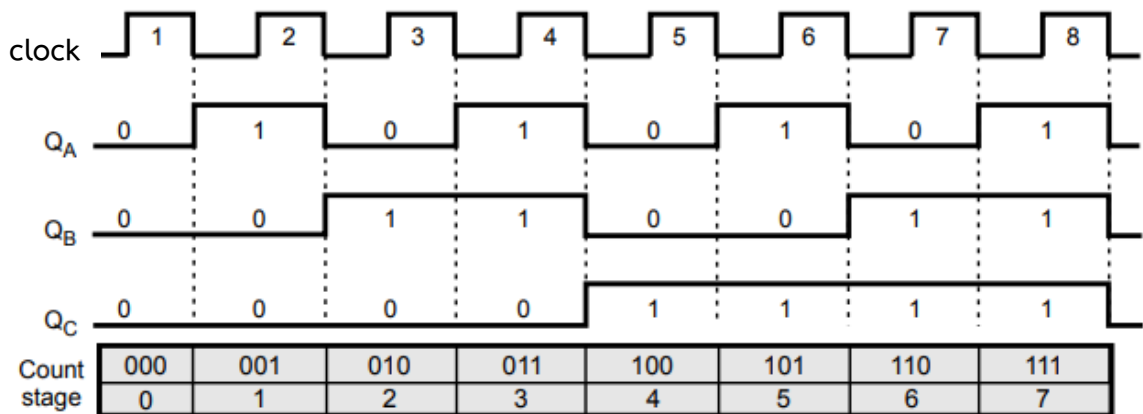
1) การนับ 0 - 7 เทียบเท่ากับเลขฐานสอง 3 บิต ($Q_C Q_B Q_A$ ตั้งแต่ 000 ถึง 111) จึงใช้ JK Flip-Flop จำนวน 3 ตัว เมื่อนับครบจะวนกลับมาเริ่มต้นใหม่โดยอัตโนมัติ โดยต่อขา $J = K = 1$ ทุกตัว ทำให้ฟลิปฟล็อปทำงานในโหมด toggle คือสลับสถานะทุกครั้งที่ได้รับขอบสัญญาณนาฬิกา

2) Asynchronous (ripple) สัญญาณนาฬิกาของแต่ละฟลิปฟล็อปจะไม่ต่อร่วมกัน แต่นำเอาเอาต์พุต Q ของฟลิปฟล็อปตัวก่อนหน้าไปเป็นสัญญาณนาฬิกาของตัวถัดไป และใช้ฟลิปฟล็อปชนิดกระตุ้นที่ขอบขาลง เพื่อให้นับขึ้น การเปลี่ยนสถานะจึงทยอยเกิดต่อเนื่องกันเป็นทอด ๆ จึงเรียกว่า ripple



รูปที่ 2 แผนภาพบล็อกของวงจรนับ 0-7 แบบ ripple (เอาต์พุต Q ของตัวก่อนหน้าเป็นสัญญาณนาฬิกาของตัวถัดไป)

3) การ MOD ความถี่ คือ เอาต์พุตแต่ละบิตมีความถี่เป็นครึ่งหนึ่งของบิตก่อนหน้า กล่าวคือ Q_A มีความถี่เป็นครึ่งหนึ่งของสัญญาณ clock, Q_B เป็นครึ่งของ Q_A , Q_C เป็นครึ่งของ Q_B เมื่อพิจารณา $Q_C Q_B Q_A$ ประกอบกันจะได้เลข 000 - 111 หรือ 0 - 7 ดูได้จากกราฟ timing



รูปที่ 3 Timing diagram

4) ข้อควรระวัง คือ propagation delay วงจรแบบ ripple มีเวลาหน่วงสะสม เมื่อนับจากค่า 7 ไปเป็น 8 ทุกบิตต้องเปลี่ยนสถานะโดยจะเปลี่ยนไล่ทีละตัว ($FF_0 \rightarrow FF_1 \rightarrow FF_2 \rightarrow FF_3$) ในระหว่างนั้นวงจรจะผ่านค่าที่ไม่ถูกต้องชั่วขณะ (glitch) ก่อนเข้าสู่ค่าที่ถูกต้อง จึงไม่เหมาะกับการใช้งานที่ความถี่สูง แต่ในการทดลองนี้ซึ่งเปลี่ยนค่าทุก 1 วินาที เวลาหน่วงดังกล่าวไม่ส่งผลต่อการทำงาน



intermediate 6,4,0 = momentary glitch before settling at 8

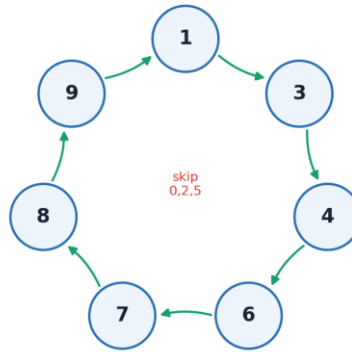
รูปที่ 4 การเกิด propagation delay จากวิธีการ ripple

5) วงจรหารความถี่ (MOD) เพื่อสร้างสัญญาณ 1 Hz สัญญาณนาฬิกาของบอร์ดมีความถี่สูงมาก (เช่น 50 MHz) จำเป็นต้องหารความถี่ลงให้เหลือ 1 Hz ด้วยวงจรนับแบบ modulo-N โดย $N = 50,000,000 / 2 = 25,000,000$ (ใช้ตัวนับประมาณ 25 บิต) เมื่อวงจรนับครบจำนวน N จะสลับสถานะเอาต์พุตหนึ่งครั้ง ทำให้ได้สัญญาณคลื่นสี่เหลี่ยมความถี่ 1 Hz สำหรับป้อนเข้าวงจรนับ

2. ให้นักศึกษาออกแบบวงจร MOD ความถี่ และวงจร Counter ขึ้นจาก 0 ไป 7 (0 1 2 3 4 5 6 7 . . .) ผ่าน JK Flip Flop แบบ Asynchronous โดยวงจร Counter จะเปลี่ยนเลขทุก ๆ 1 วินาที แล้วทดสอบการทำงานบนบอร์ด FPGA โดยแสดงผลทาง 7-Segment

หลักการวงจรนับ (Counter) โดยการกำหนดเลข แบบ Synchronous

1) ลำดับการนับ ยกตัวอย่างหากต้องการนับ 1 3 4 6 7 8 9 (วน)



รูปที่ 5 State diagram ลำดับการนับ

2) โหมดการทำงานและกฎการกระตุ้นของ JK Flip-Flop

J	K	Q
0	0	คงค่าเดิม (hold)
0	1	เป็น 0 (reset)
1	0	เป็น 1 (set)
1	1	สลับค่า (toggle)

ตารางกฎการกระตุ้น (Excitation Table) ที่ใช้ในการออกแบบพิจารณาย้อนกลับว่าเมื่อต้องการให้บิตเปลี่ยนสถานะในลักษณะใด จะต้องกำหนดค่า J และ K เป็นเท่าใด (X หมายถึง don't-care)

$Q \rightarrow Q_n$	J	K
$0 \rightarrow 0$	0	X
$0 \rightarrow 1$	1	X
$1 \rightarrow 0$	X	1
$1 \rightarrow 1$	X	0

3) สถานะที่ไม่ได้ใช้งาน กำหนดเป็น X สถานะที่ไม่ได้อยู่ในลำดับการนับ ถือเป็น don't-care โดยกำหนดค่า J และ K เป็น X ทั้งหมด ซึ่งช่วยให้สามารถจัดกลุ่มใน K-map ได้ใหญ่ขึ้น และได้สมการที่กระชับยิ่งขึ้น

4) การต่อแบบ Synchronous วงจรแบบ Synchronous ฟลิปฟล็อปทุกตัวใช้สัญญาณนาฬิกาาร่วมกัน จึงเปลี่ยนสถานะพร้อมกัน (ไม่เกิด glitch เหมือนแบบ ripple)

3. ให้นักศึกษาออกแบบวงจร Counter นับขึ้นจาก 0 ไป 9 โดยไม่เอาตัวเลขที่อยู่ในรหัสนักศึกษา 4 หลักสุดท้าย แบบ Synchronous โดยวงจร Counter จะเปลี่ยนเลขทุก ๆ 1 วินาที แล้วทดสอบการทำงานบนบอร์ด FPGA โดยแสดงผลทาง 7-Segment (เช่น รหัสนักศึกษา 64010522 จะทำการนับ 1 3 4 6 7 8 9)

3.1 สร้างตารางค่าการนับ (Output, State transition)

[illegible]

3.2 สร้างตารางค่าความจริงของอินพุต J และ K ของฟลิปฟล็อปทุกตัว โดยเติมค่าลงในตาราง (สามารถกำหนดค่า X ได้)

[illegible]

3.3 หาสมการอินพุตของฟลิปฟล็อปด้วยวิธีพีชคณิตบูลีนหรือ K-map โดยลดรูปให้ได้สมการที่กระชับที่สุด

$J_3 =$ _____ $K_3 =$ _____
 $J_2 =$ _____ $K_2 =$ _____
 $J_1 =$ _____ $K_1 =$ _____
 $J_0 =$ _____ $K_0 =$ _____

3.4 วาดไดอะแกรมของวงจร โดยใช้ JK Flip Flop มาต่อกัน และ ทดสอบการทำงานโดยการ Download ลงบอร์ด FPGA

